

QUISQUISLINGO FOR LINUX

ABOUT QUISQUISLINGO

QuisquisLingo is an offline-first language-learning application for learners and course creators.

It provides structured language courses made of Lessons, GuideBooks, Rounds, interactive exercises, review activities and Duels. Depending on the course, exercises may include text, images, recorded audio and text-to-speech.

QuisquisLingo also includes authoring tools for creating, editing, inspecting, importing and exporting courses. It is therefore intended both for people who want to study languages and for authors, teachers and other users who want to create their own learning material.

Learner progress, profiles, settings and locally created content are stored locally. QuisquisLingo does not require a user account or a permanent server connection for its normal learning functions.

The project is open source under the Mozilla Public License 2.0. Course content and other assets may have their own separate licences or rights.

Project source: <https://github.com/Quisquisnaut/QuisquisLingo>

RUNNING THE LINUX PACKAGE

Extract the complete archive before running the application.

Do not copy only the executable. The executable depends on the other files and directories contained in the package.

From a terminal opened in the extracted application directory:

```
chmod +x quisquislingo_app
./quisquislingo_app
```

The chmod command is normally required only if the executable permission was lost while downloading, copying or extracting the package.

The complete generated Linux bundle must remain together, including the executable, libraries, data files and other runtime files supplied with the package.

SYSTEM REQUIREMENTS

Requirements for the prebuilt Linux package are deliberately modest.

The package requires:

- a compatible 64-bit x86 Linux system;
- a graphical Linux desktop session;
- the normal GTK/Linux runtime libraries expected by a Flutter desktop application;
- sufficient free memory to run a desktop graphical application;
- working audio output if audio exercises or text-to-speech are used.

Flutter, Dart, Android Studio and development tools are NOT required to run the prebuilt package.

An Internet connection is not required for normal offline learning. Features that deliberately open external web resources naturally require connectivity.

TEXT-TO-SPEECH ON LINUX

QuisquisLingo uses a native Linux text-to-speech backend.

It looks for:

```
espeak-ng
```

or, as a fallback:

`espeak`

Generated speech is played through:

`aplay`

On Debian, Ubuntu, antiX and related distributions, a typical installation is:

```
sudo apt install espeak-ng alsa-utils
```

Package names may differ on other Linux distributions.

If QuisquisLingo reports that no suitable TTS voice or Linux TTS backend is available, first check that eSpeak NG and aplay are installed and accessible from PATH.

Text-to-speech is optional. QuisquisLingo can still be used without TTS, although exercises that specifically require unavailable audio may be excluded according to the application's audio settings.

Temporary WAV files generated by the Linux TTS backend are created in the system temporary directory and removed after playback.

FILES AND DIRECTORIES WRITTEN BY QUISQUISLINGO

QuisquisLingo does not need to write into the directory containing the executable during normal use.

Application data is written to locations belonging to the current Linux user.

Some paths are resolved through the operating system and Flutter and can therefore vary slightly between Linux distributions, desktop environments and user configurations.

User-visible QuisquisLingo files are generally organised below the user's application Documents location in a QuisquisLingo directory.

Depending on which features are used, this tree may include locations such as:

```
QuisquisLingo/  
  Logs/  
  Imports/  
  Exports/
```

Additional subdirectories may be created for particular functions, for example course backups or imported resources.

Course backups are stored under:

```
QuisquisLingo/Exports/Course Backups/
```

Some application preferences, learner state and other internal persistent data are stored using platform application-storage mechanisms rather than as ordinary user-editable files. Their physical Linux location is selected by the underlying platform libraries and may vary between distributions.

Temporary files may also be written to the system temporary directory when required. For example, the Linux TTS backend temporarily creates generated speech files there.

QuisquisLingo should therefore be run as the normal desktop user. It does not require root privileges for normal operation.

CRASH LOG

QuisquisLingo maintains a local crash log for diagnostic purposes.

The crash log is named:

```
quisquislingo_crash.log
```

It is stored in the application's Logs directory:

```
QuisquisLingo/Logs/quisquislingo_crash.log
```

The exact absolute path to the user's Documents location may depend on the Linux environment.

The crash log records technical information useful for diagnosing failures, including information such as:

- date and time;
- QuisquisLingo version;
- operating system information;
- architecture;
- processor count;
- locale;
- runtime information;
- error information;
- stack traces when available.

The crash logger is designed not to record course answers, profile names or other ordinary learner content.

The crash log is bounded in size so that it cannot grow indefinitely.

DIAGNOSTIC LOG

QuisquisLingo also maintains a separate diagnostic log.

The diagnostic log records selected application errors and technical diagnostic events. It is separate from the crash log.

The live diagnostic information is kept in the application's persistent settings storage. When exported by QuisquisLingo, a readable copy is created as:

```
QuisquisLingo/Logs/quisquislingo_diagnostic_log.txt
```

Diagnostic information may include:

- timestamps;
- QQL error codes;
- technical context;
- exceptions;
- stack traces;
- platform decisions such as audio or TTS availability.

Diagnostic logging is intended to help investigate application behaviour. It is not automatically uploaded to a server.

TERMINAL DIAGNOSTICS

When investigating a Linux-specific problem, it is useful to start QuisquisLingo from a terminal rather than from a graphical file manager:

```
./quisquislingo_app
```

This allows Linux runtime messages to remain visible.

Messages concerning graphics drivers, GTK, audio, TTS or other system components may appear in the terminal even when they are not part of the QuisquisLingo crash log.

To save terminal output for troubleshooting, you can use:

```
./quisquislingo_app 2>&1 | tee quisquislingo_terminal.log
```

The resulting file can be inspected or supplied with a bug report.

REPORTING A PROBLEM

When reporting a crash or Linux-specific problem, useful information includes:

- Linux distribution and version;

- desktop environment;
- QuisquisLingo version;
- what you were doing immediately before the problem;
- whether the problem can be reproduced;
- relevant terminal output;
- the QuisquisLingo crash log;
- an exported diagnostic log, when available.

Please review diagnostic files before sharing them if you have privacy concerns. QuisquisLingo's own diagnostic logging is designed to avoid ordinary learner content, but system or terminal messages can originate from external libraries or the operating system.

BUILDING QUISQUISLINGO FROM SOURCE

The complete QuisquisLingo source code is publicly available on GitHub:

<https://github.com/Quisquisnaut/QuisquisLingo>

The distributed Linux binary is therefore not the only way to run QuisquisLingo. Users and developers can download the source and build the Linux application themselves.

A current Flutter SDK and the normal Flutter Linux desktop build dependencies are required for compilation.

These normally include a compiler toolchain, CMake, Ninja, pkg-config and GTK development libraries. Exact package names depend on the Linux distribution.

Flutter can check the development environment with:

```
flutter doctor
```

A typical source build is:

```
git clone https://github.com/Quisquisnaut/QuisquisLingo.git
cd QuisquisLingo
flutter doctor
flutter pub get
flutter analyze
flutter test
flutter build linux --release
```

The resulting Linux release bundle is normally created under:

```
build/linux/x64/release/bundle/
```

Run the application from that complete bundle rather than copying only its executable.

If standard Flutter platform runner files are absent from a particular source archive, the repository also provides a Linux/macOS preparation script:

```
chmod +x tools/prepare_flutter_platforms.sh
./tools/prepare_flutter_platforms.sh
```

After preparation, repeat the normal Flutter dependency, validation and build commands.

SOURCE CODE AND LICENCE

Official repository:

<https://github.com/Quisquisnaut/QuisquisLingo>

The QuisquisLingo software source is licensed under the Mozilla Public License 2.0.

Individual courses, images, audio material and other content may be distributed under different licences or rights. Refer to the licence and attribution information supplied with the application and with individual content packages.

IMPORTANT

QuisquisLingo is under active development. Linux support should be tested on the intended distribution because graphics, audio, text-to-speech and desktop integration can differ between Linux systems.

If the application does not start, launching it from a terminal is normally the most useful first diagnostic step.